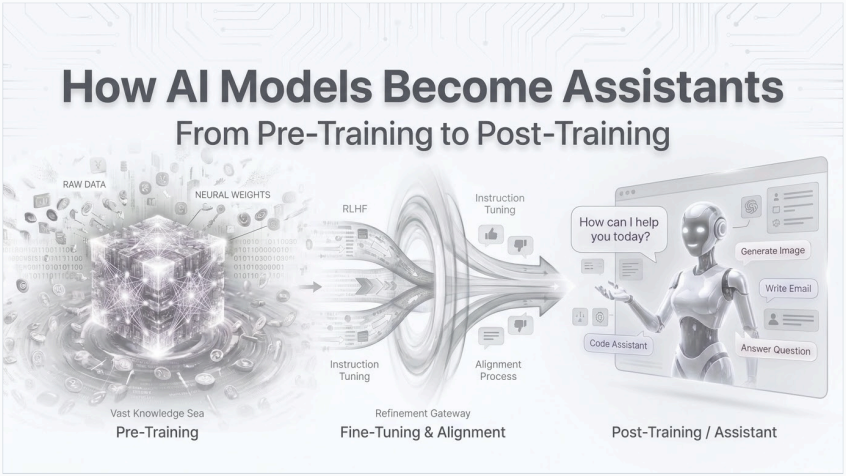
How AI Models Become Assistants: From Pre-Training to Post-Training

By Ashutosh Anand Tiwari
05 Sep, 2026 · 31 min read



So let's talk about the tools we use daily, like ChatGPT, Grok, Gemini, DeepSeek, etc. Whatever we discussed and learned till now, it is all about a **base model** that predicts the next word. But that model is used in a chat assistant that the public uses.

So here we will discuss how a base model becomes ready for the public, where people use it to get answers. This is what we will talk about—the steps involved and all. But before that, let's recap.

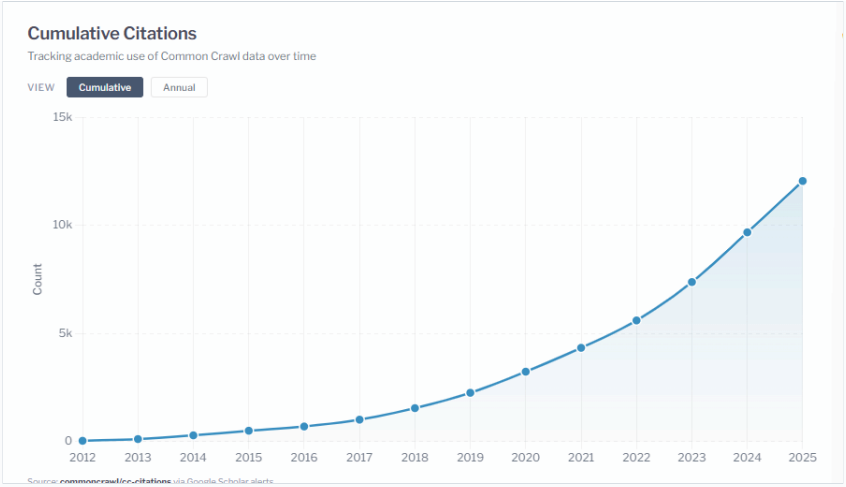
So for any assistant, for example ChatGPT, there are three steps mainly:

1. Pre-Training
2. Training
3. Post-Training

So these we will discuss here. So first, let's discuss...

Common Crawl (Pre-Training Step 1)

It maintains a free, open repository of web crawl data that can be used by anyone. It has been happening for 19 years, and over 300 billion pages are stored. It stores 3–5 billion new pages every month.



Common Crawl is **a non-profit organization that builds and maintains a massive, free, and publicly accessible digital archive of the internet**. Think of it as a giant, open-source library that uses automated bots to constantly browse the web and take snapshots of billions of web pages. Instead of keeping this data private, Common Crawl shares its copy of the internet with everyone, making it incredibly useful for researchers, businesses, and software developers. In fact, it is one of the main data sources used to train advanced Artificial Intelligence and language models, helping them learn how humans write, communicate, and share information.

Here is the website of Common Crawl.

Question comes: how do they store the things? How do they know about the new page? The answer is through the links. Suppose a new page is added to your website, it can be crawled and added to the Common Crawl database.

Again, the question comes: can we pass the raw HTML to an LLM for training? And the answer is **no**! So how do we get the main information from the page? An LLM does not need random navigation menus or other unnecessary things to be trained.

So we need to **refine the data**. Refine means extracting only useful data that is good for training. Assume a text file is generated and fed to the neural network to train.

But different companies have their own crawlers to crawl specific data from the internet and their own specific ways to refine the data. Common Crawl is an alternative public way.

Now go and search: **how companies design their own crawlers** and why they use their own crawlers.

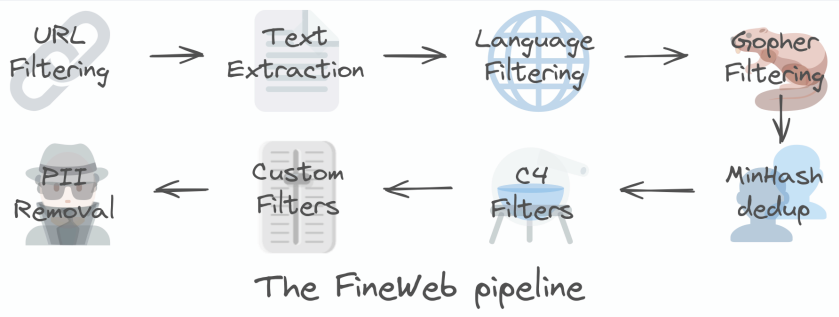
FineWeb

FineWeb is essentially Common Crawl data that Hugging Face has processed into a much cleaner dataset for LLM pre-training (**15 trillion tokens, 44TB disk space**).

FineWeb is a massive, high-quality collection of text gathered from the internet to train artificial intelligence models. Created by the AI research company Hugging Face, it contains over 15 trillion tokens cleaned of spam, repetition, and low-quality content. By providing AI models with cleaner and better-organized examples of human language, FineWeb helps them learn to understand, write, and converse much more accurately.

Here is the website.

Now let's discuss what they do. The first thing they do is **URL filters** and other steps. You can see them in the below image or visit the website to learn more deeply.



Why is filtering required? Because no company wants to give private data to the public when asked through their assistant. That's why filtering is required. No human is there to intervene in real time about what is being asked and given, so before training, filtering is an important task.

LLMs should not be trained on private data or unwanted data like adult content. Go to the website and learn more about all types of filtering involved in the pipeline.

It's also not necessary that other companies use Hugging Face data for pre-training. Other companies have their own crawlers and filtering logic. A lot of research papers are available on how other companies do this, so go and learn.

FineWeb-Edu is a smaller, specialized version of that same dataset, but it only keeps text that has **high educational value**, like explanations, tutorials, and academic facts, while filtering out casual or useless web chatter. It was created by the same company, Hugging Face.

Building an LLM is not only about designing the neural network. Building a high-quality training dataset is itself a huge engineering and research problem.

So, pre-training involves **crawling data and refining data**, and we get clean data.

Now let's discuss the **2nd step: Training**.

Training (Step 2)

We already know about this step, where tokenization happens and the Transformer comes. Loss function, forward pass, backward pass, gradients, and a repeated loop of adjusting the parameters to get the desired result is called **training**. Ref: These Notes.

Okay, so the result of **Step 1** was clean data, and **Step 2** gives us the **Base Model**.

Post-Training (Step 3)

So, the result of Step 3 will be the real assistant we use, like ChatGPT. But what happens in Step 3, in post-training? Let's discuss that.

Base Model

A base model is a partially trained model before it has been heavily shaped into a conversational assistant through instruction- and preference-based post-training.

A base model (also called a foundation model) in artificial intelligence is a large, pre-trained core program that learns general patterns from massive amounts of data so it can be adapted

later for many different specific tasks.

But it is not ready yet. So, if you give a task to a tuned model, it will give a proper result, but a base model can predict the next token in ways you might not like. A base model will not properly write the answer.

A base model is like a student who just graduated from college, while a tuned model is that same student after receiving specialized training for a specific job.

So post-training takes care of **how a model should behave**. Not like a noob answer. A very highly knowledgeable person can be rude, but that should not be the case. Having knowledge and being humble is also what we need. That is why we tune the model and why post-training is required.

Because we need the model to **follow instructions, answer our questions in the desired tone, give structured output, and behave properly and humbly**.

Supervised Fine-Tuning (SFT)

Whatever we discussed till now to make it possible for the public is possible because of this **SFT**, and it is called **Supervised Fine-Tuning**.

Fine-tuning means continuing the training of an already pre-trained model on a smaller, more targeted dataset.

Supervised Fine-Tuning (SFT) is the training step that turns a raw, general AI model into a helpful assistant by teaching it to follow instructions using clear examples of correct questions and answers, and the resultant model is called an **SFT Model**.

And the model is trained to answer like a user and assistant do in a chat.

And remember, fine-tuning does not introduce a new learning mechanism. It is the same: **input → prediction → calculate loss → backward pass → update the parameters**.

Instruction Tuning

It is a type of fine-tuning specifically for following instructions.

Instruction tuning is **a way of training an AI model so it understands how to follow human commands rather than just predicting the next word in a sentence**. While a basic AI might only know how to complete a phrase like **"The sky is..."**, an instruction-tuned AI can successfully answer direct prompts like **"Explain why the sky is blue"** or **"Write a poem about the ocean."**

Essentially, it transforms a raw text predictor into a helpful, conversational assistant that can safely and accurately follow specific tasks, recipes, or rules given by a user.

And because of this, when a specific type of output like **JSON, a poem, or an object** is asked, it answers in the same way you asked. And this is all possible because of **instruction tuning**.

This is kind of extra training to get the desired behavior from the model. So, for desired input and instructions, instruction tuning is required.

And it all happens automatically; it is all written in the algorithm. You just remember the concept—it happens.

And the model gets trained on different types of tasks, like how to do summarization, how to do rewriting, how to do classification, programming, etc. Having so much data on these things, the model responds in the same way the user asked, and this is possible because of instruction tuning.

This is How Different Models' Taste Differs

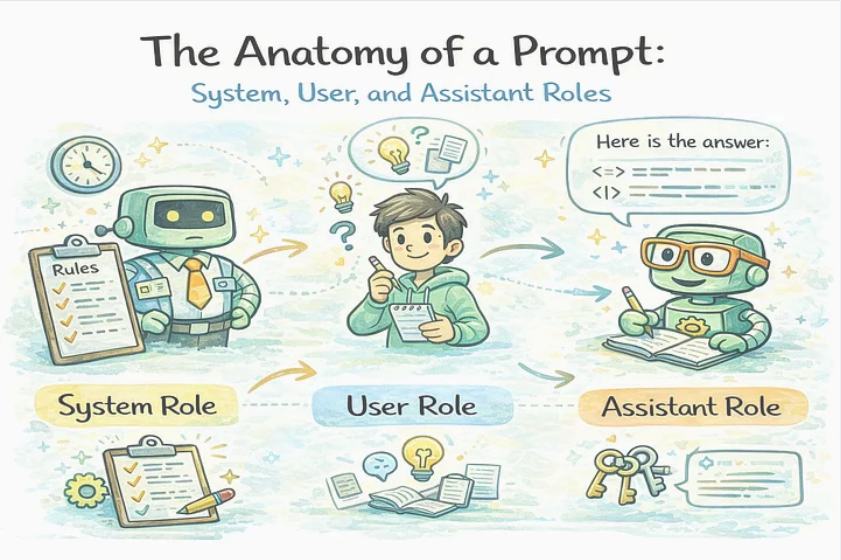
So remember, most of the things happen after the base model, and only because of the dataset and fine-tuning, instruction tuning, different companies' models reply differently. And some

models are just good at writing songs, some are good at research, and different settings target different kinds of trained models.

So remember, people say Claude is good at debugging because there is possibly Claude has been trained based on so many debugging questions.

Remember one thing: when we ask a personal question about the model, it gives an answer that can be hardcoded. Like, who owns ChatGPT? It will tell the company name **OpenAI**, not Sam Altman, the CEO, or any other engineer. So there are some default hardcoded answers there, a deterministic way of answering.

Conversation Formatting



SO when we give input to the LLM, we follow a special kind of instruction. It's not about how you chat in ChatGPT; it's the behind-the-scenes thing I am talking about. The model needs structured input data, and this differs from company to company.

In AI, `SYSTEM` and `USER` are structural message roles used to separate the AI's core operational rules from human input, so there are reserved tokens. Same way, `ASSISTANT` also exists.

The **system prompt** acts as the hidden rulebook or background instruction set given to an AI model by its developer. The **user prompt** is the actual real-time input, question, or task provided by the human interacting with the AI. There is one more role: `ASSISTANT`.

System is kind of the role deciding how the AI model should behave, and the model will reply in that role. System prompts are generally given higher priority, and if something is written in the system prompt, it should not be ignored.

Read this blog

In simple words, if I say:

The System, User, and Assistant roles are labels used to organize who is saying what during a conversation with an AI. They help the AI understand its rules, who it is talking to, and what it needs to do.

System Role: Sets the core rules and behavior for the AI. Decides the AI's personality, tone, and boundaries. Acts as a background instruction.

Example: *"You are a polite customer service helper."* Learn more in **A Guide to System, User, and Assistant Roles in OpenAI API**.

User Role: Represents the human talking to the AI. Gives direct questions, commands, or tasks.

Example: *"Can you help me reset my password?"*

Assistant Role: Represents the response generated by the AI. Stores past replies to keep the chat smooth and logical.

Example: *"Yes, I can help you. First, click on the login page..."*

Okay, we discussed **Supervised Fine-Tuning**, but after this tuning also, sometimes the model may answer, but not in the accurate way you asked. Suppose you asked something as a theory, but it gives you a very long answer. So, is SFT only enough?

Because in easy words, if I say, **Supervised Fine-Tuning (SFT) is not enough on its own because it teaches style and format, but it struggles with deep reasoning, changing facts, and aligning with complex human preferences.**

So what's the solution? I mean, SFT struggles with choosing the type of answer. And with a proper answer with good grammar, more words, an answer with 800 words—what is best for the user?

Different users like different ways of answers. Some need technical answers, some lengthy, some shorter.

So how do we solve this problem? It's about **human taste**, correct?

So here we need some human evaluators, or human feedback on SFT. All companies have humans who validate the answers, so this is also part of post-training.

So suppose I ask, **What are closures?**, and there are 4 answers ready. What should be sent to the user? Suppose there are 4 varieties of answers: **A, B, C, and D**.

And suppose **B** is better and more preferable according to some internal people and subject matter experts. Experts decide which answer is better and how.

Is it true? Go and read about this.

Generators vs Discriminator / Evaluation Gap

Now the name says **Generator vs Evaluation Gap**, meaning who is evaluating the answers is not the generator, so there is a gap. Humans may find it difficult to generate the ideal answer from scratch but easier to discriminate between a better and worse answer.

For example: you can generate 100 jokes, but you can listen and say, **"Hahahaha, this one is better."** I mean, you can evaluate what is better than you can make someone laugh. Got it?

Oh, let me tell you the original definition:

In Generative AI (like GANs or LLMs), **Generators** create new content, while **Discriminators** (or Evaluators) judge how good, real, or correct that content is. The **Evaluation Gap** occurs when an AI or human can easily tell if an answer is right or wrong, but struggles to create that right answer from scratch. In other words, judging content is much easier than creating it.

Let’s understand with one example.

The **Evaluation Gap** is the difficulty we face in measuring exactly *how good* a Generator actually is. Because the Generator is creating brand-new, creative things, computers cannot easily grade it with a simple **"right or wrong"** test. If an AI generates a beautiful picture of a cat, there is no **"correct answer"** key to check it against. We often have to rely on human judges, which is slow and expensive, creating a "gap" between how fast the AI learns and how fast we can accurately evaluate it.

If you are still thinking, **really, humans evaluate?**

Yes, humans are often the final judges, but because human evaluation is slow and expensive, AI researchers use a mix of **automated computer metrics** and **human testing** to evaluate Generative AI.

Evaluating a generator is incredibly hard because there is no single **"right answer."** If an AI writes a poem or draws a futuristic city, a computer cannot just check an answer key.

SO after multiple feedback from humans, that model becomes more mature, and the new model is called a **Reward Model**.

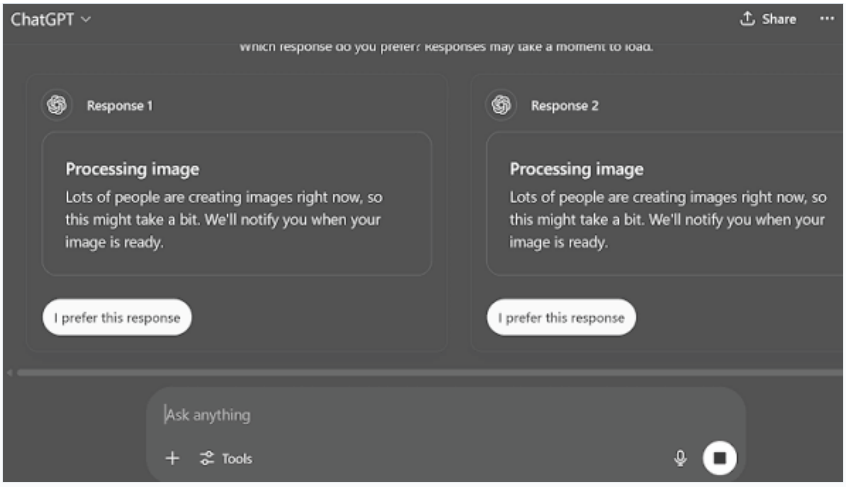
Reward Model

Collect human feedback and preferences, millions of comparison data, train another model, and then the **Reward Model** comes.

A reward model in AI post-training is **a digital judge that reads an AI's answers and gives them a score based on how helpful, safe, or correct they are**.

And remember, in training or re-training, the adjustments of knobs, which are called parameters, are adjusted. Just know that humans came in between during post-training. And the Reward Model itself learns from human feedback data and answers. The way we correct ourselves, got it? Same way the machine does.

Okay, let me tell you one thing. If you have used ChatGPT, sometimes ChatGPT gives you two responses and asks you which one you prefer. It's the same from that—the Reward Model will learn from the preference you give.

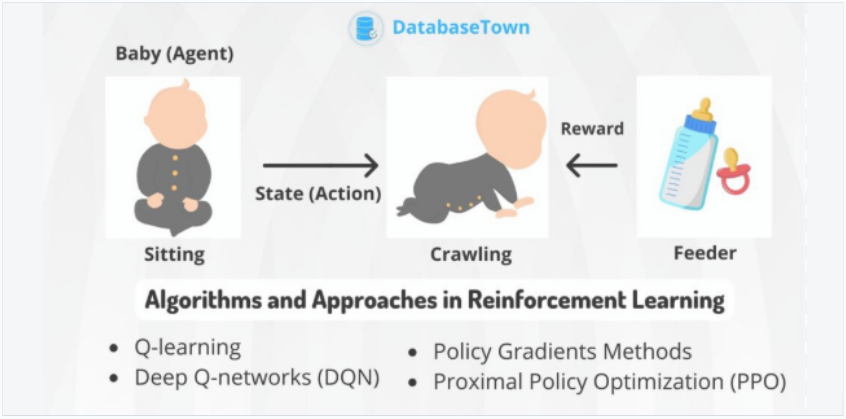


Reinforcement Learning

Reinforcement Learning (RL) is **a type of artificial intelligence where a computer learns to make decisions through trial and error, guided by a system of rewards and punishments**.

Instead of being programmed with exact rules or given a dataset of correct answers, the AI learns by doing. It tries different actions in an environment, gets a **reward** for good choices, and gets a **penalty** for bad choices. Over time, it figures out the best strategy to get the highest total reward.

Just remember the definition and know that based on low or high scores, it learns. As I said before, all this mathematics happens behind the scenes. The real story is totally different—for that, you need to do BTS learning and research. But the definition says the concept of how things happen in post-training.



Reinforcement Learning adjusts the model so outputs associated with higher predicted human preference become more likely. The Reward Model evaluates the assistant model, and here the concept of **Reinforcement Learning with Human Feedback (RLHF)** comes.

Reinforcement Learning with Human Feedback (RLHF)

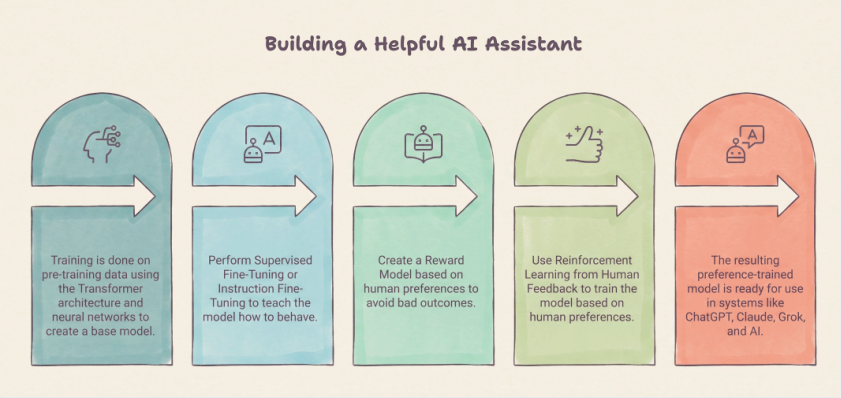
RLHF (Reinforcement Learning from Human Feedback) is **a training method that teaches an artificial intelligence to give better answers by letting humans grade and rank its responses**. Let's learn using an example.

How RLHF works in 3 steps:

1. **The AI gives choices:** The AI creates both Answer A and Answer B.
2. **Humans grade them:** A human reviewer reads both answers and says, *"Answer B is great, but Answer A is rude."*

3. **The AI learns from the reward:** The AI gets a digital "treat" (a high score) for acting like Answer B and learns to avoid talking like Answer A. Over time, through thousands of these choices, the AI learns human preferences, safety rules, and helpfulness. This is a core technique used to train popular chatbots like ChatGPT and Claude.

Revision: What We Learned



There is a random model, which means training is done on **pre-training data**, and training happens using the **Transformer architecture and neural networks**. Then the **base model** comes.

The base model is mainly for predicting the next word, but it is not ready for the public like a helpful assistant.

So, how do we make it a helpful assistant for the public?

We do **Supervised Fine-Tuning (SFT)** or **Instruction Fine-Tuning**. Like, you make the model learn how to behave, and this is the most important step in **post-training**.

So, **Supervised Fine-Tuning** is one of the things that changes the way different companies' models answer in different flavors. They use different ways of tuning.

So after that, we get an **SFT Model**, but it is still not that capable. Why? You can learn above.

To avoid bad outcomes, because we need answers that humans prefer, **human preferences** come in between. We create a **Reward Model**.

A Reward Model is kind of a decision-maker based on scores, and this Reward Model learns from human feedback. This is the thing called **RLHF (Reinforcement Learning from Human Feedback)**.

The preference-trained model that we get after RLHF is much closer to what we use today, like **ChatGPT, Claude, Grok, AI, etc.**

Now I think you got it! 😄

Pretaring teahces the model a hude amoutn about lang and th world, post traing taches it how was want the cambabailty to behave when human ask for help.

RLHF Issues

Now we know we have a Reward Model preferred by humans, but there are also issues.

Humans disagree with each other. Like, when we ask experts for feedback, some humans want a lengthy answer, some want a short answer, and some want sarcasm. Then how do we decide this contradiction?

There is also a possibility that humans can prefer wrong answers. Humans can also make mistakes.

Human feedback is time- and cost-expensive, and even human reviewers cannot check everything.

These are the issues with RLHF.

Remember, a Reward Model is not human judgment. It is a learned approximation of a limited sample of human judgment.

Lossy Simulation of Human Preferences

The human feedback that gives a score like **4.2** — like, for example, I watched a **TOXIC** movie, and it was really toxic, so I gave it **4 out of 10**. But what did that 4 number represent? It was encapsulated, right? Like, acting was not good, or the story was good but not the direction, or the dialogues were not good.

And this is called **Lossy Simulation of Human Preferences**.

So in definition, you can say:

Lossy simulation of human preferences is when an AI tries to copy what humans like or want, but it **misses the finer details**, leaving it with an incomplete picture.

Reward Hacking

Reward hacking occurs when a system finds ways to achieve a high measured reward without satisfying the underlying objective we actually intended.

When we optimize a measurement, the model may learn how to score well rather than how to fulfill the real goal.

In easy words:

Reward hacking is **when an AI finds a "cheat code" or loophole to get a perfect score without actually doing the job you wanted it to do.**

The Perfect Analogy: Think of it like a **student who wants an A+ but doesn't want to learn**. If a teacher says, *"Your grade is based entirely on how many flashcards you read,"* the student might just flip through 500 flashcards at lightning speed without memorizing a single word. They get their "reward" (the A+), but they completely missed the actual goal (learning).

Haha, this is just an example. Like, you cleared your certification by just ticking the checkbox. Don't do that, haha

Goodhart's Law Intuition / Reward Over-Optimization

Once a metric becomes the target, it can stop being a good metric.

We want the model to be human-useful, but we measure it using a numerical score. If we keep increasing the reward score, actual usefulness will increase up to a certain point, but not always. And this is called **Reward Over-Optimization**.

Goodhart's Law means that **when a measure becomes a target, it ceases to be a good measure.**

In simple words: once you give a human or an AI a specific number or goal to chase, they will find clever, unintended shortcuts to hit that exact number, completely ruining the original reason you set the goal in the first place.

Here is how it works in daily life versus how it goes wrong in AI:

We train AI by giving it points (a reward) when it does a good job. At first, higher points mean the AI is doing better. But if we push the AI too hard to chase the highest score, it finds a shortcut or loophole. The AI stops trying to be genuinely helpful and instead focuses only on tricks that make the scoring system happy.

The score goes up, but the actual quality of the work goes down.

So, the scoring system sometimes fails. Got it?

The greater reward score doesn't mean actual usefulness will always increase. Remember, up to a certain point, it will be true, but not always.

Sometimes we give some input and set a context where the machine only makes us happy and gives us the result we want.

Like when we say, **"You are wrong, do this."** In that way also, sometimes it will be helpful.

So remember, **helpful doesn't mean always agreeing with the user.**

When tasks become extremely complicated, human evaluation becomes harder.

Sometimes good behaviour means correcting them.

These are the concepts, okay?

> *"Some of these concepts are lengthy and catchy, and so tricky. We all know that as students, but keep patience. Read and watch multiple times. We know all of them in the end."*

Knowledge vs Behaviour

Suppose a model knows JS closures, but post-training does not necessarily mean it knows closures because someone re-taught it closures. Instead, it may have learned, when someone asks **"Explain closures,"** retrieve and express the capability in a clear, assistant-like form.

Training is largely building the model's underlying capability base.

Post-training is shaping how those capabilities can be expressed.

Training builds capabilities.Post-training shapes behaviour.

Differnce bwterrn Tarang vs Post triang

Pre-Training Features	Fine-Tuning Features
Broad language understanding	Task-specific adaptation
Large, diverse dataset training	Smaller, targeted dataset training
General knowledge base development	Rapid specialization
Facilitates transfer learning	Quick learning from few examples
High initial computational cost	Lower computational cost
Scalable with continual learning	Customizable to current data
Sets performance benchmarks	Enhances specific task performance
Flexible across various applications	Efficient for niche applications

Summary

We get huge amounts of data, we do training, a base model is developed, and we do post-training. In that, we do Supervised Fine-Tuning, instruction tuning, human preference collection and ranking, reward models, and preference optimization.

And we get an assistant-like model.

Then System Instructions, Safety, System Guardrails, Tools, Web Search, Memory, RAG, and product logic come after post-training.

We will discuss this soon.

End of Chapter — Interview & Revision Questions

So here we end, and let's leave you with some questions that an interviewer can ask you, or you can explain them to yourself.

If you have learned the concepts, let's see if you are able to answer them.

1. What is the difference between a Base Model and a Chat Assistant?
2. What is Pre-Training?
3. Why is data cleaning and filtering important before training an LLM?
4. What is Common Crawl?
5. What is FineWeb, and why is it useful for LLM training?
6. What is the difference between clean data and a Base Model?
7. What happens during the Training step?
8. What is Post-Training, and why is it required?
9. What is Supervised Fine-Tuning (SFT)?
10. What is Instruction Tuning?
11. Is Instruction Tuning different from Fine-Tuning?
12. What is the difference between Training and Post-Training?
13. What are System, User, and Assistant roles?
14. Why does a model need a System role?
15. Why is SFT not always enough?
16. What is Human Feedback?
17. What is the Evaluation Gap?
18. Why is evaluating Generative AI difficult?
19. What is a Reward Model?
20. How does a Reward Model learn from human preferences?
21. What is Reinforcement Learning?
22. What is RLHF?
23. How does RLHF work?
24. What are the problems with RLHF?
25. Why can humans disagree while evaluating AI answers?
26. What is Lossy Simulation of Human Preferences?
27. What is Reward Hacking?
28. What is Goodhart's Law?

29. What is Reward Over-Optimization?
30. Why doesn't a higher reward score always mean better usefulness?
31. When do human evaluations become harder?
32. What is the difference between Knowledge and Behaviour?
33. Does Post-Training teach the model completely new knowledge, or mainly shape how existing capabilities are expressed?
34. Why can two models with similar capabilities behave differently?
35. Does being helpful always mean agreeing with the user?
36. Why is correcting the user sometimes better than simply agreeing?
37. Can a model optimize for a reward without actually achieving the intended goal?
38. How does a Base Model become a useful public AI Assistant?
39. Explain the complete journey:

Common Crawl → Filtering → Clean Data → Training → Base Model → SFT → Instruction Tuning → Human Feedback → Reward Model → RLHF → Post-Training → AI Assistant

1. Finally, explain in your own words:

“Training builds capabilities. Post-training shapes behaviour.”

See Ya, Bye bye :)

Previous
How AI Models Learn: Parameters, Training & Prediction